

FORMATION EMBARQUÉE

Module 06 — Autres langages

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 06 — Les autres langages et outils indispensables de l'embarqué

Autour du trio C/C++/VHDL gravitent des outils qu'on croise dans tout projet réel : l'assembleur (pour comprendre), MicroPython (pour prototyper), Rust (l'avenir proche), Make/CMake (pour construire), un RTOS (pour structurer), Linux embarqué et le shell (pour les grosses cibles).

1. Assembleur : lire plus qu'écrire

On n'écrit presque plus d'assembleur, mais **savoir le lire** est irremplaçable : déboguer un crash, optimiser une routine, comprendre ce que le compilateur fait vraiment.

1.1 Le modèle : registres + instructions

Un CPU ARM Cortex-M a 16 registres (R0–R12, SP pile, LR retour, PC compteur de programme). Une instruction = une opération élémentaire :

```
; ARM Thumb – équivalent de : c = a + b;
LDR  r0, =var_a      ; r0 ← adresse de a
LDR  r1, [r0]         ; r1 ← contenu à cette adresse (charge a)
LDR  r0, =var_b
LDR  r2, [r0]
ADD  r3, r1, r2       ; r3 ← r1 + r2
LDR  r0, =var_c
STR  r3, [r0]         ; mémorise le résultat dans c
```

Architecture **load/store** : on charge en registre, on calcule, on range.

1.2 Ce qu'il faut savoir faire

- Reconnaître : **MOV**, **LDR/STR**, **ADD/SUB**, **CMP** + branchements (**BEQ**, **BNE**, **B**), **PUSH/POP**, **BL** (appel de fonction).
- Comprendre la **convention d'appel** ARM : arguments dans R0–R3, retour dans R0 — c'est ce qui rend C et assembleur interopérables.
- Générer et lire l'assembleur de ton propre C :

```
arm-none-eabi-gcc -S -Os main.c -o main.s    # voir ce que produit le compilateur
arm-none-eabi-objdump -d firmware.elf        # désassembler un binaire
```

Exercice royal : compiler une fonction simple avec **-O0** puis **-O2** et comparer — tu ne verras plus jamais l'optimiseur de la même façon.

2. MicroPython / CircuitPython : prototyper en minutes

Python interprété *sur* le microcontrôleur (ESP32, Raspberry Pi Pico, cartes PyBoard). Idéal pour valider un capteur ou une idée avant de coder en C.

```
# Raspberry Pi Pico – blink + capteur, en 10 lignes
from machine import Pin, ADC, I2C
import time

led = Pin(25, Pin.OUT)
pot = ADC(26)

while True:
    led.toggle()
    print("pot =", pot.read_u16())
    time.sleep(0.5)
```

- **REPL** sur le port série : tu tapes du code *en direct* sur la carte.
- Outils : Thonny (IDE débutant), `mpremote`, VS Code.
- Limites : 10–100× plus lent que le C, RAM consommée par l'interpréteur, latences non déterministes → pas pour le temps réel dur.
- Python côté PC est de toute façon indispensable : scripts d'analyse de logs, génération de tables, bancs de test (pyserial pour parler à tes cartes), traitement des mesures (matplotlib, pandas).

3. Rust embarqué : la sécurité mémoire sans ramasse-miettes

Rust apporte les garanties de Java (pas de pointeur fou, pas de course de données) **sans machine virtuelle ni GC** — vérifié à la compilation par le *borrow checker*. Adopté par Linux (noyau), l'automobile et l'industrie.

```
// Blink sur STM32 avec le HAL embassy/stm32 (représentatif)
#![no_std]                // pas de bibliothèque standard : bare-metal
#![no_main]

use embassy_stm32::gpio::{Level, Output, Speed};
use embassy_time::Timer;

#[embassy_executor::main]
async fn main(_spawner: embassy_executor::Spawner) {
    let p = embassy_stm32::init(Default::default());
    let mut led = Output::new(p.PC13, Level::High, Speed::Low);

    loop {
        led.toggle();
        Timer::after_millis(500).await; // async SANS OS : tâches coopératives
    }
}
```

Concepts propres à Rust : **ownership** (chaque valeur a un unique propriétaire), **borrowing** (`&` , `&mut` vérifiés), `Result<T, E>` au lieu d'exceptions, `unsafe` pour les rares accès registres bruts (isolés et audités). Apprends-le **après** le C : tu comprendras *pourquoi* chaque règle existe. Point d'entrée : *The Rust Book* puis *The Embedded Rust Book* (gratuits en ligne).

4. Make, CMake et la ligne de commande

4.1 Makefile : l'automatisation historique

```
CC      = arm-none-eabi-gcc
CFLAGS  = -mcpu=cortex-m3 -mthumb -Os -Wall -Wextra
SRCS    = main.c uart.c capteur.c
OBJS    = $(SRCS:.c=.o)

firmware.elf: $(OBJS)
    $(CC) $(CFLAGS) $^ -T stm32.ld -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

flash: firmware.elf
    openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
        -c "program firmware.elf verify reset exit"

clean:
    rm -f $(OBJS) firmware.elf
```

Une règle = **cible** : **dépendances** + commandes (indentées par une **tabulation**). Make ne reconstruit que ce qui a changé.

4.2 CMake : le standard moderne

```
cmake_minimum_required(VERSION 3.20)
project(firmware C)
add_executable(firmware main.c uart.c capteur.c)
target_compile_options(firmware PRIVATE -Wall -Wextra -Os)
```

CMake *génère* des Makefiles (ou Ninja). C'est l'outil des projets STM32Cube, ESP-IDF, Zephyr, Pico SDK — incontournable en entreprise.

4.3 Le shell et Git

Le quotidien de l'embarqué se passe dans un terminal :

```
ls, cd, cat, grep -r "motif" src/, find . -name "*.c"
picocom -b 115200 /dev/ttyUSB0          # console série (ou minicom, screen)
dmesg | tail                            # "quel port série ma carte a-t-elle pris ?"
hexdump -C firmware.bin | head          # inspecter un binaire

git init / add / commit / push          # versionne TOUT, dès le premier projet
git diff / log / branch / merge
```

5. FreeRTOS : structurer le firmware en tâches

Quand la super-boucle + FSM ne suffit plus (plusieurs activités à échéances différentes), un **RTOS** ordonnance des tâches par priorité, de façon préemptive et déterministe. FreeRTOS est le plus répandu (et intégré à l'ESP32 d'office).

```
void tache_capteur(void *param) {
    for (;;) {
        mesure_t m = lire_capteur();
        xQueueSend(file_mesures, &m, portMAX_DELAY); // vers l'autre tâche
        vTaskDelay(pdMS_TO_TICKS(100));             // dort 100 ms (le CPU est libre)
    }
}

void tache_affichage(void *param) {
    mesure_t m;
    for (;;) {
        if (xQueueReceive(file_mesures, &m, portMAX_DELAY) == pdTRUE)
            afficher(m);
    }
}

int main(void) {
    file_mesures = xQueueCreate(16, sizeof(mesure_t));
    xTaskCreate(tache_capteur, "capteur", 256, NULL, 2, NULL);
    xTaskCreate(tache_affichage, "affich", 512, NULL, 1, NULL);
    vTaskStartScheduler(); // ne revient jamais
}
```

Concepts à maîtriser : tâche, priorité, préemption, **file (queue)**, **sémaphore**, **mutex** (et l'inversion de priorité), taille de pile par tâche. Alternatives : **Zephyr** (le RTOS qui monte, style Linux) et l'approche async de Rust/embassy.

6. Linux embarqué (aperçu)

Dès que la cible est costaud (Raspberry Pi, passerelles, box), on embarque un vrai Linux :

- **Espace utilisateur** : tes programmes C/C++/Python accèdent au matériel via `/dev` (`/dev/ttyUSB0` , `/dev/i2c-1` , `/dev/spidev0.0` , `gpiod`).
- **Device Tree** : description du matériel fournie au noyau.

- **Yocto / Buildroot** : usines à images Linux sur mesure.
- **Drivers noyau** : modules en C — le sommet de la programmation bas niveau.

C'est un domaine d'emploi énorme (« BSP engineer », « Linux embedded »). Y aller après avoir bien pratiqué C + shell.

7. Quel langage pour quel travail ? (synthèse du cours)

Besoin	Outil
Firmware micro, driver, RTOS	C (puis Rust)
Application embarquée riche (ESP32, STM32, Linux)	C++
Prototype rapide, banc de test, scripts	Python / MicroPython
Logique matérielle, FPGA	VHDL (ou Verilog)
Supervision, Android, back-end IoT	Java (ou Kotlin/C#)
Automates industriels	IEC 61131-3 (modules 07-08)
Comprendre / déboguer au plus bas	Assembleur (lecture)
Construire, flasher, automatiser	Make/CMake + shell + Git

Exercices

1. Compile une fonction C (`-O0` vs `-O2`), désassemble, et explique deux optimisations que tu observes.
2. Écris un Makefile pour un projet C à 3 fichiers avec cibles `all` , `clean` , et reconstruction minimale.
3. Sur ESP32 (Arduino + FreeRTOS intégré) : deux tâches — l'une clignote une LED à 2 Hz, l'autre imprime un compteur chaque seconde — communiquant par une queue.
4. En MicroPython sur Pico (ou simulateur Wokwi) : refais la station météo du module 03 et compare le temps de développement avec la version C++.

➡ Suite : [Module 07 — Suite Siemens](#) : on entre dans l'industrie.